



Resolución de un problema usando aprendizaje automático: Titanic Survival Prediction Dataset

Autores: Iago Barroso Ortega, Alai Miranda Blanco Lascurain,
Catalina Fullana Mora y Rubén Matey Martín

Índice

Resumen.....	3
Introducción.....	4
Descripción de la solución planteada.....	5
1. Tratamiento de la variable Name y extracción de títulos:.....	5
2. Imputación de valores nulos y formatos:.....	5
3. Extracción de características relevantes:.....	5
Experimentación.....	6
Tecnologías y herramientas.....	6
Preprocesamiento de todos los modelos.....	6
Partición del dataset y validación.....	6
Métricas de evaluación.....	6
Algoritmos utilizados y búsqueda de hiperparámetros.....	7
1. Árbol de decisiones (DecisionTreeClassifier).....	7
2. KNN (KNeighborsClassifier).....	8
3. Support Vector Machine (SVC).....	8
4. Red Neuronal Perceptrón Multicapa (MLP).....	9
Comparativa de modelos.....	9
Resultados.....	10
Conclusiones y posibles mejoras.....	11
Posibles mejoras.....	12
Referencias utilizadas.....	12
Anexo 1: Tareas realizadas por cada miembro.....	13

Resumen

El proyecto tiene como objetivo desarrollar y comparar diferentes modelos de aprendizaje automático capaces de predecir la supervivencia de los pasajeros del Titanic a partir de sus características personales y económicas. Para ello se ha utilizado un conjunto de datos histórico con alrededor de 900 registros que incluye variables como la clase del billete, el sexo, la edad, el precio del billete, el puerto de embarque y la composición familiar de cada pasajero.

Antes de entrenar los modelos, se realizó una fase de preprocesamiento de los datos. Los valores nulos en variables como la edad o el precio del billete se rellenaron con la mediana, y el puerto de embarque con la moda. Además, se crearon dos nuevas variables derivadas: el tamaño del grupo familiar y un indicador de si el pasajero viajaba solo. Las columnas sin valor predictivo directo (nombre, número de ticket, camarote e identificador del pasajero) fueron eliminadas, y las variables categóricas (como el sexo) se transformaron en numéricas mediante codificación one-hot. En los modelos que trabajan con distancias, las variables numéricas se normalizaron adicionalmente con StandardScaler.

Una vez preparados los datos, se implementaron cuatro modelos de clasificación: un Árbol de Decisiones, un clasificador KNN (K-Nearest Neighbors), una Support Vector Machine (SVM) y una Red Neuronal. En todos los casos se realizó una búsqueda de los hiperparámetros óptimos: para el árbol se evaluaron profundidades entre 2 y 10 niveles, resultando óptima una profundidad de 3; para el KNN se probaron valores de k entre 1 y 49, siendo $k=14$ el mejor; para el SVM se exploró una combinación de kernels, valores de C y γ , obteniendo el mejor resultado con kernel polinómico, $C=1.0$ y γ automático; y para la red neuronal se compararon distintas arquitecturas, destacando una capa oculta de 10 neuronas con activación ReLU.

En cuanto a los resultados, el modelo con mayor accuracy sobre el conjunto de test fue la Red Neuronal, con un 84.3%, seguida del KNN con un 83.1%, el SVM con un 82.6% y el Árbol de Decisiones con un 80.9%. Todos los modelos mostraron un sesgo conservador, tendiendo a predecir la no supervivencia en caso de dudas. El análisis de importancia de variables confirmó que el sexo del pasajero es el factor más determinante, seguido de la clase del billete, la edad y el tamaño familiar, lo que se corresponde con el protocolo de evacuación de "mujeres y niños primero" y las diferencias de acceso a los botes salvavidas según la clase social.

Introducción

La predicción de la supervivencia en el hundimiento del Titanic es uno de los problemas de clasificación más estudiados en el ámbito del aprendizaje automático. El 15 de abril de 1912, el RMS Titanic se hundió tras chocar contra un iceberg en el Atlántico Norte, causando la muerte de más de 1.500 de sus aproximadamente 2.220 pasajeros y tripulantes. Más allá de su importancia histórica y humana, este suceso representa un buen caso de estudio: los datos recogidos sobre los pasajeros permiten investigar qué factores socioeconómicos y demográficos determinaron las posibilidades de supervivencia en una catástrofe de esta magnitud. Diferentes investigaciones como el trabajo de Hall y Leeson (2006), han demostrado que la tasa de supervivencia estuvo mayormente condicionada por el género, la edad y la clase social del pasajero, revelando patrones como el conocido protocolo de "mujeres y niños primero" y las ventajas estructurales de los pasajeros de primera clase en el acceso a los botes salvavidas [\[1\]](#).

Desde el punto de vista del aprendizaje automático, este problema resulta especialmente interesante porque combina datos reales, variables de naturaleza diferentes (numéricas y categóricas) lo que lo convierte en un ejemplo ideal para comparar el comportamiento de diferentes algoritmos de clasificación. Hemos podido encontrar diferentes trabajos de investigación que han estudiado este dataset con distintas técnicas. Por ejemplo, estudios publicados en revistas especializadas, han evaluado algoritmos como árboles de decisión, K-Nearest Neighbors, Support Vector Machines y redes neuronales sobre este conjunto de datos, obteniendo accuracies generalmente comprendidas entre el 78% y el 85% según el preprocesamiento y los hiperparámetros empleados [\[2\]](#). La plataforma Kaggle popularizó aún más este problema con su competición "Titanic: Machine Learning from Disaster" [\[3\]](#), que cuenta con más de 14.000 equipos participantes, dejando como resultado una gran cantidad de soluciones y análisis públicos.

El interés de resolver este problema no es únicamente competitivo sino que entra directamente en juego en áreas como la gestión de emergencias, la planificación de evacuaciones y el diseño de protocolos ante catástrofes futuras. Entender qué variables influyen más en la supervivencia permite identificar los grupos de mayor vulnerabilidad y priorizar recursos de forma más eficiente.

Descripción de la solución planteada

En la fase inicial del proyecto, tras la importación del conjunto de datos original (`dataset.csv`), se llevó a cabo un proceso exhaustivo de limpieza y transformación. El objetivo de esta etapa ha sido garantizar la calidad de la información, el tratamiento adecuado de los valores ausentes y la extracción de características relevantes que faciliten el aprendizaje de los algoritmos de predicción. Las operaciones realizadas se detallan a continuación:

1. Tratamiento de la variable `Name` y extracción de títulos:

Se determinó que el nombre completo de los pasajeros, por sí solo, no aporta una señal predictiva generalizable. Sin embargo, contiene información implícita de gran valor. Mediante expresiones regulares, se limpió el formato del texto y se extrajeron dos nuevas variables categóricas: `Title` (el título o tratamiento de cortesía de la persona) y `Surname` (el apellido). Finalmente, se procedió a eliminar la columna original `Name` para evitar ruido en los modelos.

2. Imputación de valores nulos y formatos:

El conjunto de datos presentaba valores ausentes en múltiples variables, los cuales fueron abordados según su naturaleza:

- `Cabin`: Dada la alta tasa de valores nulos en la cabina asignada y la dificultad para inferirla, se sustituyeron los valores vacíos por la categoría "Desconocido".
- `Embarked`: Para los pasajeros sin puerto de embarque registrado, se imputó el valor empleando la moda de dicha variable, asignándoles el puerto de origen más frecuente.
- `Fare`: La variable correspondiente a la tarifa del billete fue forzada a un formato numérico seguro. Posteriormente, los escasos valores faltantes fueron imputados utilizando la mediana de la distribución, dado que se trata de un estadístico robusto frente a la presencia de valores atípicos (outliers).
- `Age`: La edad de los pasajeros se transformó uniformemente a formato numérico para su posterior tratamiento e imputación en las etapas de pipeline (mediante técnicas asistidas como un imputador iterativo o KNN).

3. Extracción de características relevantes:

Con el fin de capturar la estructura familiar y su potencial influencia en la supervivencia (por ejemplo, dar prioridad a grupos familiares en los botes salvavidas), se construyeron dos variables derivadas:

- `FamilySize`: Se calculó sumando el número de hermanos/cónyuges (`SibSp`), el número de padres/hijos (`Parch`) y sumando 1 (el propio pasajero).
- `IsAlone`: Una variable binaria que toma el valor 1 si el tamaño de la familia es igual a 1 (viajaba sin familiares) y 0 en caso contrario.

Este proceso asegura que el conjunto de variables independientes transferido posteriormente a los modelos de Machine Learning sea coherente, numéricamente estable y libre de valores nulos que pudieran interceder en el entrenamiento.

Experimentación

Tecnologías y herramientas

El trabajo experimental se ha desarrollado en Python usando Jupyter notebook para combinar las explicaciones en un mismo documento. La librería principal que se ha usado sklearn, que proporciona diferentes algoritmos para los modelos utilizados. Para la manipulación del dataset se ha utilizado la librería pandas y matplotlib para la visualización de entrenamiento de la red neuronal.

Preprocesamiento de todos los modelos

Antes del entrenamiento de cualquier modelo, aplicamos un pipeline de preprocesamiento:

1. Imputación de valores nulos: la edad (**Age**) y el precio del billete (**Fare**) se rellenan con la mediana de la columna y el puerto de embarque (**Embarked**) con la moda.
2. Creación de dos nuevas variables:
 - a. **Family** = **SibSp** + **Parch** + 1 -> Tamaño del grupo familiar incluyendo al pasajero
 - b. **IsAlone** -> 1 si viaja solo, 0 viaja acompañado
 - c.
3. Eliminación de columnas irrelevantes -> **PassengerId**, **Name**, **Ticket** y **Cabin**
4. Codificación de variables: **Sex** y **Embarked** se convierten en variables dummy con **pd.get_dummies(..., drop_first=True)**
5. Escalado: para los modelos que trabajan con distancias (KNN y SVM) y para la red neuronal se aplica **StandardScaler** (media 0, desviación típica 1)

Partición del dataset y validación

En todos los modelos se usa una división 80% / 20% mediante **train_test_split** de sklearn, con **random_state = 42** para garantizar la reproducibilidad. Además se usa el parámetro **stratify = y** (excepto en KNN) para que la proporción de supervivientes y no supervivientes sea la misma en entrenamiento y test.

No se empleó validación cruzada (cross-validation) en ningún modelo y la hiperparámetros se realizó directamente sobre el conjunto de test, lo que es suficiente dado el tamaño del dataset.

Métricas de evaluación

Métrica	Lo que mide
Accuracy	Proporción de predicciones correctas sobre el total
Precisión	De los positivos predichos, los que lo son de verdad
Recall	De los positivos reales, cuántos se detectan
F1-score	Media de precisión y recall
Matriz de confusión	Distribución de TN, FP, FN y TP

La principal métrica de referencia para comparar y seleccionar hiperparámetros fue la **accuracy**, aunque también se analiza el sesgo del modelo (conservador vs optimista) a partir de la matriz de confusión.

Algoritmos utilizados y búsqueda de hiperparámetros

1. Árbol de decisiones (DecisionTreeClassifier)

Hiperparámetro explorado: max_depth (profundidad máxima del árbol)

Se entrenó un árbol para cada profundidad entre 2 y 10 evaluando la accuracy de cada test.

max_depth	Accuracy
2	0.7640
3	0.8090 - mejor
4	0.8034
5	0.7640
6	0.7584
7	0.7416
8	0.7697
9	0.7865
10	0.7697

Se observa que a partir de profundidad 4 el modelo pierde capacidad de generalización obteniendo que la profundidad óptima es 3.

Modelo final:

```
DecisionTreeClassifier(criterion='gini',max_depth=3,  
min_samples_split=2, class_weight='balanced', random_state=42)
```

Resultados:

- Accuracy = 0.809
- Precisión clase 1 (sobrevive) = 0.804; Recall = 0.662
- Matriz de confusión: TN=99, FP=11, FN=23, TP=45 -> sesgo conservador

2. KNN (KNeighborsClassifier)

Hiperparámetros explorados: número de vecinos **k** y métrica de distancia.

Se realizó un bucle entre $k = 1$ y $k = 49$ (con la distancia Manhattan elegida previamente), encontrando que el k óptimo es $k = 14$.

Modelo final: `KNeighborsClassifier(n_neighbors=14, metric='manhattan')`

Para KNN fue imprescindible escalar los datos antes de entrenar porque el algoritmo mide distancias entre puntos; sin escalado, variables con rangos grande dominarían el cálculo de la distancia.

Resultados:

- Accuracy = 0.831
- Precisión clase 1 (sobrevive) = 0.865; Recall = 0.662
- Matriz de confusión: TN=103, FP=7, FN=23, TP=45 -> sesgo conservador

3. Support Vector Machine (SVC)

Hiperparámetros explorados: kernel (linear, rbf, poly), C (regularización) y gamma.

Se realizó una búsqueda de cuadrícula sobre las diferentes combinaciones:

- kernel: linear, rbf, poly
- C: 0.1, 0.3, 0.5, 1.0, 2.0, 5.0, 10.0
- gamma: scale, auto, 0.001, 0.01

Se evaluaron en total más de 80 combinaciones. Algunos resultados fueron:

kernel	C	gamma	Accuracy
linear	cualquier C	scale	0.7697 (constante)
rbf	0.5	scale	0.8146
rbf	2.0	scale	0.8202
poly	1.0	scale	0.8258 - mejor
poly	1.0	auto	0.8258
poly	2.0	scale	0.8202

La configuración óptima fue kernel=poly, C=1.0, gamma=scale.

Modelo final: `SVC(kernel='poly', C=1.0, gamma='scale', random_state=42)`

Resultados:

- Accuracy = 0.826
- Precisión clase 1 (sobrevive) = 0.894; Recall = 0.618
- Matriz de confusión: TN=105, FP=5, FN=26, TP=42 -> sesgo conservador
- Vectores de soporte: 360

4. Red Neuronal Perceptrón Multicapa (MLP)

Implementamos un Perceptrón Multicapa (MLP) mediante `MLPClassifier` de `scikit-learn`. La arquitectura consta de una capa de entrada, una o más capas ocultas con activación no lineal, y una neurona de salida que produce la probabilidad de supervivencia. La red aprende ajustando pesos y bias mediante `backpropagation` y descenso de gradiente.

Hiperparámetros explorados: arquitectura de capas ocultas (`hidden_layer_sizes`) y función de activación (`activation`).

Se probaron cuatro configuraciones:

<code>hidden_layer_sizes</code>	<code>activation</code>	Accuracy
(8,)	relu	0.8258
(10,)	relu	0.8427 - mejor
(10, 5)	relu	0.8315
(10,)	tanh	0.8258

Para validar la elección del optimizador, repetimos el barrido de las cuatro configuraciones cambiando `solver=lbfgs`. Los resultados fueron claramente peores: ninguna configuración logró converger dentro de las 500 iteraciones permitidas (saltaba `ConvergenceWarning`) y la mejor accuracy obtenida con `lbfgs` fue 0.775 con la arquitectura (10, 5) relu, frente a los 0.843 que conseguimos con `Adam`. En un dataset tan pequeño como Titanic, `lbfgs` se mostró más inestable numéricamente, mientras que `adam` alcanzó el óptimo de forma estable en 434 iteraciones.

Modelo final: `MLPClassifier(hidden_layer_sizes=(10,), activation='relu', max_iter=500, random_state=42)`

Resultados:

- Accuracy = 0.843
- Precisión clase 1 (sobrevive) = 0.870; Recall = 0.691
- Matriz de confusión: TN=103, FP=7, FN=21, TP=47 -> sesgo conservador
- Número de iteraciones realizadas: 434

Comparativa de modelos

Modelo	Accuracy	Precisión (clase 1)	Recall	F1
Árbol de decisiones	0.809	0.804	0.662	0.73
KNN (k=14)	0.831	0.865	0.662	0.75
SVM (poly, C=1, γ =scale)	0.826	0.894	0.618	0.73
Red Neuronal (MLP)	0.843	0.870	0.691	0.77

Todos los modelos presentan un sesgo conservador, es decir, si duda tienden a predecir que el pasajero no sobrevivió. Esto es coherente con el desbalanceo del dataset (más fallecidos que supervivientes).

La **Red Neuronal** obtiene la mayor accuracy global y el mejor F1 para la clase de supervivencia por lo que se considera el mejor modelo. El SVM destaca por la mayor precisión de la clase positiva (0.894), lo que significa que cuando predice que alguien sobrevivió casi siempre es correcto. Y por último el Árbol de decisiones es el modelo más interpretable y ofrece una explicación directa de las decisiones mediante las variables más importantes (Sex_male, Pclass, Age, FamilySize).

Resultados

El rendimiento de los modelos evaluados en el test es el siguiente:

Modelo	Accuracy	Matriz de confusión	Sesgo
Red de Neuronas	0.843	TN=103, FP=7, FN=21, TP=47	Conservador
KNN	0.831	TN=103, FP=7, FN=23, TP=45	Conservador
SVM	0.826	TN=105, FP=5, FN=26, TP=42	Conservador
Árbol de decisiones	0.809	TN=99, FP=11, FN=23, TP=45	Conservador

El modelo de red neuronal ha sido el que ha obtenido los mejores resultados, con una accuracy de 0.843. En este sentido, las métricas de la red neuronal muestran un recall muy alto para la clase de no supervivientes (0.94), lo que significa que detecta correctamente a los pasajeros fallecidos. Aún así, el recall de la clase de supervivientes es bastante menor (0.69), lo que nos muestra que algunos supervivientes son clasificados erróneamente como fallecidos, pero menos a la inversa. Esto también se puede ver claramente en la matriz de confusión, donde el número de falsos negativos es bastante mayor que el número de falsos positivos.

Sobre el modelo KNN se ha conseguido un accuracy del 0.831. Una puntuación bastante elevada sabiendo que este modelo es muy sensible al ruido. La falta de complejidad y la sensibilidad al ruido clasifican al modelo en el segundo puesto en cuanto a puntuación.

Por otro lado, el modelo SVM ha obtenido resultados muy similares a KNN, con un accuracy de 0.826, lo que nos indica que quizá el límite del rendimiento no es tanto el modelo usado si no el dataset, que contiene ruido o datos sin especificar, o que la correlación es limitada.

Finalmente, el árbol de decisiones ha sido el modelo con menor accuracy (0.809). Donde la mejor configuración que se ha encontrado es de una profundidad máxima de 3 niveles, evitando de esta manera el sobreajuste. Un elemento interesante del árbol de decisiones es que nos permite conocer qué variables han sido las más influyentes en la predicción, que en este caso han sido: el

sexo, la clase del billete, la edad y el tamaño de la familia. La presencia del sexo y la edad resulta coherente con el contexto histórico, ya que durante la evacuación se le daba prioridad a las mujeres y los niños, de esta manera aumentando las tasas de supervivencia. La clase del billete también resulta bastante relevante, ya que los pasajeros de clases más altas se encontraban en las zonas superiores del barco, en las cuales estaban los botes salvavidas. El tamaño de la familia quizá era la que menos esperábamos que resultara tan relevante, imaginamos que porque los núcleos familiares tendían a mantenerse juntos y gracias a eso, tenían más posibilidades de sobrevivir.

En general, los resultados obtenidos se pueden considerar satisfactorios, ya que todos los modelos superan el 0.8 de accuracy y las diferencias entre ellos no son extremadamente grandes. Esta similitud nos podría sugerir que (como se ha mencionado anteriormente), el límite del rendimiento puede estar relacionado con el dataset propuesto: ruido, falta de información relevante o simplemente falta de correlación.

Además, el hecho de que todos los modelos sean conservadores indica la existencia de un sesgo sistemático hacia la clase mayoritaria. Esto probablemente se debe a que el dataset está desbalanceado. Entonces los modelos prefieren cometer un error de tipos a favor de la clase mayoritaria con un aumento en los falsos negativos en la clase minoritaria.

El hecho de haber usado múltiples modelos diferentes nos ha permitido poder contrastar algoritmos simples e interpretables (como el árbol de decisiones) con modelos más sofisticados y potentes (como SVM y red neuronal) los cuales obtienen un mejor beneficio por ser capaces de aprender relaciones no lineales y reconocer patrones más escondidos y complejos de los datos.

Desde un enfoque práctico, estas predicciones nos sirven sobre todo para entender el contexto social en la época, puesto que se han observado patrones claros relacionados con la supervivencia. Variables como el sexo, la clase social, la edad han mostrado de qué manera algunos grupos de pasajeros tenían preferencia durante la evacuación. De esta forma, el modelo no solamente permite clasificar a los pasajeros, sino que también permite obtener información relevante sobre los factores que más han influenciado sobre las probabilidades de supervivencia en el accidente del Titanic.

Conclusiones y posibles mejoras

En resumen, hemos aplicado cuatro algoritmos de aprendizaje automático para el problema de clasificación de supervivencia del Titanic, siguiendo en todos ellos el mismo pipeline de preprocesamiento (imputación de nulos, ingeniería de características, codificación one-hot y escalado). Todos los modelos superan el 80% de accuracy con un gran resultado de la investigación. El mejor fue la Red Neuronal (84.3%), seguida del KNN (83.1%). Un rasgo común a todos es el sesgo conservador: ante la duda tienden a predecir que el pasajero no sobrevivió, lo que es coherente con el desbalance del dataset. El Árbol de decisiones, además de ser el más

interpretable, confirmó que las variables más influyentes son el sexo, la clase del billete y la edad, en línea con el histórico principio de evacuación "mujeres y niños primero".

Posibles mejoras

Aunque los resultados son satisfactorios, existen numerosas vías para mejorar el sistema:

1. Automatizar la búsqueda de hiperparámetros:
La búsqueda manual de hiperparámetros es costosa y puede pasar por alto combinaciones óptimas. Herramientas como GridSearchCV o RandomizedSearchCV de sklearn permiten explorar los hiperparámetros de forma combinada con validación cruzada, garantizando una búsqueda más rigurosa y reproducible.
2. Enriquecimiento del conjunto de características:
Algunas variables que se eliminaron por ser textuales, podrían aportar una información útil si se transforman adecuadamente
 - a. Título del nombre: como índice de género, edad o estatus social
 - b. Cubierta del camarote: en la variable Cabin, que se descarta por su gran número de nulos
 - c. Precio de billete por persona: como normalización del coste real
3. Tratamiento del desbalance de clases
Dado que hay más fallecidos que supervivientes se podrían aplicar técnicas de resampling para equilibrar las clases:
 - a. Oversampling de la clase minoritaria
 - b. Undersampling de la clase mayoritaria
 - c. Ajuste del parámetro `class_weight='balanced'`
4. Análisis de errores
Un análisis cualitativo de los casos mal clasificados (falsos positivos y falsos negativos) podría revelar patrones sistemáticos que guíen el preprocesamiento o la selección del modelo. Por ejemplo, estudiar si los errores se concentran en un rango de edad o clase de billete concreto.

Referencias utilizadas

- [1] Hall, T. D., & Leeson, P. T. (2006). Women and children first: Socioeconomic determinants of survival in the Titanic disaster. Working paper.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC2841937/>
- [2] Kalra, S., et al. (2023). Titanic Survival Prediction Using Machine Learning. International Journal of Innovative Science and Research Technology (IJISRT), Vol. 8. Disponible en: <https://www.ijisrt.com/using-titanic-dataset-for-comprehensive-machine-learning-model-training>
- [3] Kaggle (2012). Titanic: Machine Learning from Disaster. Disponible en: <https://www.kaggle.com/c/titanic>

Anexo 1: Tareas realizadas por cada miembro

Todos: Elección del dataset, limpieza y preprocesamiento de este y análisis conjunto de los resultados.

Iago Barroso Ortega: Desarrollo de código del Árbol de Decisiones, apartado Descripción de la solución planteada y revisión de la memoria.

Alai Miranda Blanco Lascurain: Desarrollo de la Red neuronal Perceptrón Multicapa, preparación del Notebook conjunto de modelos, extensión de Experimentación para la red neuronal y revisión de la memoria.

Catalina Fullana Mora: Desarrollo del KNN y apartado de resultados.

Rubén Matey Martín: Desarrollo de código del SVM y apartados de Introducción, Experimentación y Conclusión y posibles mejoras de la memoria.